Contents

System Design Complete Guide **5**
Everything You Need to Ace System Design Interviews . . . 5

Table of Contents **5**

1. Introduction to System Design **6**
What is System Design? . . . 6
Key Characteristics of Distributed Systems . . . 6
1. Scalability . . . 6
2. Reliability . . . 6
3. Availability . . . 6
4. Efficiency . . . 6
5. Manageability . . . 7

2. Scalability **7**
Vertical Scaling (Scale Up) . . . 7
Horizontal Scaling (Scale Out) . . . 7
Scaling Strategies . . . 7

3. Load Balancing **7**
What is Load Balancing? . . . 7
Load Balancing Algorithms . . . 7
1. Round Robin . . . 7
2. Weighted Round Robin . . . 8
3. Least Connections . . . 8
4. Weighted Least Connections . . . 8
5. IP Hash . . . 8
6. Least Response Time . . . 8
7. Random . . . 8
Types of Load Balancers . . . 8
Layer 4 (Transport Layer) . . . 8
Layer 7 (Application Layer) . . . 8
Health Checks . . . 8
High Availability for Load Balancers . . . 9

4. Caching **9**
What is Caching? . . . 9
Cache Locations . . . 9
1. Client-Side Cache . . . 9
2. CDN Cache . . . 9
3. Application Cache . . . 9
4. Database Cache . . . 9
Caching Strategies . . . 9
1. Cache-Aside (Lazy Loading) . . . 9
2. Read-Through Cache . . . 10
3. Write-Through Cache . . . 10
4. Write-Behind (Write-Back) Cache . . . 10
5. Write-Around Cache . . . 11
Cache Eviction Policies . . . 11
LRU (Least Recently Used) . . . 11
LFU (Least Frequently Used) . . . 11
FIFO (First In, First Out) . . . 11
TTL (Time To Live) . . . 11
Random . . . 11
Cache Invalidation . . . 11

Distributed Caching . . . 11
Redis . . . 11
Memcached . . . 11
Redis vs Memcached . . . 12

5. Database Design **12**
Database Types . . . 12
Relational Databases (RDBMS) . . . 12
NoSQL Databases . . . 12
Database Indexing . . . 13
B-Tree Index . . . 13
Hash Index . . . 13
Composite Index . . . 13
Full-Text Index . . . 13
Bitmap Index . . . 13
Normalization . . . 13
First Normal Form (1NF) . . . 13
Second Normal Form (2NF) . . . 13
Third Normal Form (3NF) . . . 13
Denormalization . . . 14

6. SQL vs NoSQL **14**
When to Use SQL . . . 14
When to Use NoSQL . . . 14
Comparison Table . . . 14
ACID vs BASE . . . 14
ACID (SQL) . . . 14
BASE (NoSQL) . . . 14

7. Database Scaling **15**
Replication . . . 15
Master-Slave (Primary-Replica) . . . 15
Master-Master (Multi-Primary) . . . 15
Sharding (Horizontal Partitioning) . . . 15
Sharding Strategies . . . 15
Sharding Challenges . . . 16
Federation (Functional Partitioning) . . . 16

8. Consistent Hashing **16**
The Problem with Traditional Hashing . . . 16
How Consistent Hashing Works . . . 16
Adding/Removing Servers . . . 17
Virtual Nodes . . . 17
Use Cases . . . 17

9. CAP Theorem **17**
Definition . . . 17
Why Not All Three? . . . 17
System Classifications . . . 17
CP Systems (Consistency + Partition Tolerance) . . . 17
AP Systems (Availability + Partition Tolerance) . . . 18
CA Systems (Consistency + Availability) . . . 18
PACELC Theorem . . . 18

10. Message Queues **18**
What are Message Queues? . . . 18
Benefits . . . 18

Messaging Patterns . . . 18
1. Point-to-Point (Queue) . . . 18
2. Publish-Subscribe (Topic) . . . 18
3. Request-Reply . . . 18
Message Queue Technologies . . . 19
Apache Kafka . . . 19
RabbitMQ . . . 19
Amazon SQS . . . 19
Redis Pub/Sub . . . 19
Delivery Guarantees . . . 19
At-Most-Once . . . 19
At-Least-Once . . . 19
Exactly-Once . . . 19
Message Queue Design Considerations . . . 19

11. Microservices Architecture **20**
Monolithic vs Microservices . . . 20
Monolithic . . . 20
Microservices . . . 20
Microservices Principles . . . 20
Service Communication . . . 20
Synchronous (Request/Response) . . . 20
Asynchronous (Event-Driven) . . . 20
API Gateway . . . 20
Service Discovery . . . 21
Client-Side Discovery . . . 21
Server-Side Discovery . . . 21
Circuit Breaker Pattern . . . 21
Saga Pattern . . . 21
Choreography . . . 21
Orchestration . . . 22
Microservices Challenges . . . 22

12. API Design **22**
REST (Representational State Transfer) . . . 22
REST Principles . . . 22
HTTP Methods . . . 22
REST Best Practices . . . 22
GraphQL . . . 23
GraphQL vs REST . . . 23
gRPC . . . 24
gRPC Features . . . 24
When to Use gRPC . . . 24
API Authentication . . . 24
API Keys . . . 24
OAuth 2.0 . . . 24
JWT (JSON Web Tokens) . . . 24

13. Content Delivery Networks (CDN) **25**
What is a CDN? . . . 25
How CDN Works . . . 25
CDN Benefits . . . 25
CDN Content Types . . . 25
Static Content . . . 25
Dynamic Content . . . 25
CDN Caching Strategies . . . 25
Push CDN . . . 25

Pull CDN . . . 25
CDN Providers . . . 26
CDN Configuration . . . 26

14. Proxies **26**
Forward Proxy . . . 26
Reverse Proxy . . . 26
Common Reverse Proxies . . . 26
Nginx . . . 26
HAProxy . . . 26
Envoy . . . 27

15. Storage Systems **27**
Block Storage . . . 27
Object Storage . . . 27
Object Storage Features . . . 27
File Storage . . . 27
Comparison . . . 27
Distributed File Systems . . . 27
HDFS (Hadoop Distributed File System) . . . 27
GlusterFS . . . 28

16. Distributed Systems Concepts **28**
Consensus Algorithms . . . 28
Paxos . . . 28
Raft . . . 28
Raft States . . . 28
Leader Election . . . 28
Heartbeat . . . 28
Quorum . . . 28
Vector Clocks . . . 29
Gossip Protocol . . . 29
Two-Phase Commit (2PC) . . . 29
Three-Phase Commit (3PC) . . . 29

17. Security **29**
Authentication vs Authorization . . . 29
HTTPS/TLS . . . 29
Common Security Practices . . . 30
1. Input Validation . . . 30
2. Encryption . . . 30
3. Authentication . . . 30
4. Authorization . . . 30
5. Rate Limiting . . . 30
6. Logging and Monitoring . . . 30
OAuth 2.0 Flows . . . 30
Authorization Code Flow . . . 30
Implicit Flow . . . 30
Client Credentials Flow . . . 30
Resource Owner Password Flow . . . 30

18. Monitoring and Logging **31**
The Three Pillars of Observability . . . 31
1. Metrics . . . 31
2. Logs . . . 31
3. Traces . . . 31
Key Metrics (RED Method) . . . 31

Key Metrics (USE Method) . . . 31
Alerting . . . 31
Log Aggregation Architecture . . . 32

19. Rate Limiting **32**
Why Rate Limiting? . . . 32
Rate Limiting Algorithms . . . 32
1. Token Bucket . . . 32
2. Leaky Bucket . . . 32
3. Fixed Window Counter . . . 32
4. Sliding Window Log . . . 33
5. Sliding Window Counter . . . 33
Rate Limiting Strategies . . . 33
User-based . . . 33
IP-based . . . 33
Endpoint-based . . . 33
Tiered . . . 33
Implementation . . . 33
HTTP Headers . . . 33

20. Back-of-the-Envelope Calculations **34**
Important Numbers to Know . . . 34
Storage . . . 34
Time . . . 34
Latency Numbers . . . 34
Traffic Estimates . . . 34
Quick Conversions . . . 34
Example Calculation: Twitter-like System . . . 34

21. System Design Interview Framework **35**
Step 1: Requirements Clarification (5 minutes) . . . 35
Functional Requirements . . . 35
Non-Functional Requirements . . . 35
Constraints . . . 35
Step 2: Back-of-Envelope Estimation (5 minutes) . . . 35
Step 3: High-Level Design (10 minutes) . . . 35
Step 4: Deep Dive (15 minutes) . . . 35
Step 5: Identify Bottlenecks (5 minutes) . . . 35
Common Mistakes . . . 35
Interview Tips . . . 36

Quick Reference Tables **36**
Choosing the Right Database . . . 36
Choosing the Right Cache . . . 36
Choosing Communication Pattern . . . 36

System Design Complete Guide

Everything You Need to Ace System Design Interviews

Table of Contents

1. Introduction to System Design
2. Scalability

3. Load Balancing
4. Caching
5. Database Design
6. SQL vs NoSQL
7. Database Scaling
8. Consistent Hashing
9. CAP Theorem
10. Message Queues
11. Microservices Architecture
12. API Design
13. Content Delivery Networks (CDN)
14. Proxies
15. Storage Systems
16. Distributed Systems Concepts
17. Security
18. Monitoring and Logging
19. Rate Limiting
20. Back-of-the-Envelope Calculations
21. System Design Interview Framework

---

1. Introduction to System Design

What is System Design?

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves making high-level decisions about:

- **Architecture patterns** (monolithic vs microservices)
- **Data storage solutions** (SQL, NoSQL, caches)
- **Communication protocols** (REST, GraphQL, gRPC)
- **Scalability strategies** (horizontal vs vertical)
- **Reliability and fault tolerance**

Key Characteristics of Distributed Systems

1. Scalability

The capability of a system to handle growing amounts of work by adding resources.

2. Reliability

The probability that a system will perform its required functions under stated conditions for a specified period.

3. Availability

The percentage of time that a system is operational and accessible.

```
Availability = Uptime / (Uptime + Downtime)

99% availability = 3.65 days downtime/year
99.9% (three 9s) = 8.76 hours downtime/year
99.99% (four 9s) = 52.56 minutes downtime/year
99.999% (five 9s) = 5.26 minutes downtime/year
```

4. Efficiency

Measured by latency (response time) and throughput (bandwidth).

5. Manageability

How easy it is to operate and maintain the system.

---

2. Scalability

Vertical Scaling (Scale Up)

Adding more power to existing machines (CPU, RAM, Storage).

Pros: - Simple to implement - No code changes required - No distributed system complexity

Cons: - Hardware limits - Single point of failure - Expensive (non-linear cost) - Downtime during upgrade

Horizontal Scaling (Scale Out)

Adding more machines to the system.

Pros: - No hardware limits - Better fault tolerance - Cost-effective (commodity hardware) - No downtime for scaling

Cons: - More complex architecture - Data consistency challenges - Network overhead - Requires load balancing

Scaling Strategies

```
+------------------+------------------+------------------+
|     Layer        |    Strategy      |    Examples      |
+------------------+------------------+------------------+
| Application      | Horizontal       | Add app servers  |
| Database         | Replication      | Read replicas    |
| Database         | Sharding         | Partition data   |
| Cache            | Distributed      | Redis Cluster    |
| Storage          | Object Storage   | S3, GCS          |
+------------------+------------------+------------------+
```

---

3. Load Balancing

What is Load Balancing?

Load balancing distributes incoming network traffic across multiple servers to ensure no single server bears too much load.

```
                    +---> Server 1
                    |
Client --> Load --> +---> Server 2
           Balancer |
                    +---> Server 3
```

Load Balancing Algorithms

1. Round Robin

Distributes requests sequentially across servers.

```
Request 1 -> Server 1
Request 2 -> Server 2
Request 3 -> Server 3
Request 4 -> Server 1 (cycle repeats)
```

Pros: Simple, equal distribution **Cons:** Doesn't consider server load or capacity

2. Weighted Round Robin

Servers with higher weights receive more requests.

```
Server 1 (weight=3): Gets 3 requests
Server 2 (weight=2): Gets 2 requests
Server 3 (weight=1): Gets 1 request
```

3. Least Connections

Routes to server with fewest active connections.

Pros: Better for varying request processing times **Cons:** Requires connection tracking

4. Weighted Least Connections

Combines least connections with server weights.

5. IP Hash

Routes based on client IP hash for session persistence.

```
hash(client_ip) % num_servers = server_index
```

Pros: Session affinity without storing session data **Cons:** Uneven distribution if clients have similar IPs

6. Least Response Time

Routes to server with lowest response time and fewest connections.

7. Random

Randomly selects a server.

Types of Load Balancers

Layer 4 (Transport Layer)

- Operates on TCP/UDP
- Faster (no content inspection)
- Based on IP and port

Layer 7 (Application Layer)

- Operates on HTTP/HTTPS
- Content-based routing
- Can inspect headers, cookies, URLs
- SSL termination

Health Checks

Load balancers periodically check server health:

```
Active Health Checks:
- HTTP GET /health
- TCP connection test
- Custom scripts

Passive Health Checks:
- Monitor actual traffic
```

```
- Track error rates
- Response time monitoring
```

High Availability for Load Balancers

```
        +---> Active LB ---> Servers
        |         |
Client -+    (heartbeat)
        |         |
        +---> Passive LB --> Servers
              (standby)
```

---

4. Caching

What is Caching?

Caching stores copies of frequently accessed data in a faster storage layer to reduce latency and database load.

Cache Locations

```
Client --> CDN --> Load Balancer --> App Server --> Cache --> Database
  |         |                            |
  v         v                            v
Browser   Edge                       Application
Cache     Cache                         Cache
```

1. Client-Side Cache

- Browser cache
- Mobile app cache
- HTTP caching headers

2. CDN Cache

- Geographic distribution
- Static content
- Edge servers

3. Application Cache

- In-memory (Redis, Memcached)
- Local cache (process memory)

4. Database Cache

- Query cache
- Buffer pool

Caching Strategies

1. Cache-Aside (Lazy Loading)

Application manages the cache.

```
Read:
1. Check cache
2. If miss, read from DB
```

```
3. Store in cache
4. Return data

Write:
1. Write to DB
2. Invalidate/update cache
```

```javascript
async function getData(key) {
    // Check cache first
    let data = await cache.get(key);

    if (data === null) {
        // Cache miss - read from database
        data = await database.get(key);

        // Store in cache with TTL
        await cache.set(key, data, TTL);
    }

    return data;
}
```

Pros: - Only requested data is cached - Cache failures don't break the system

Cons: - Cache miss penalty (3 operations) - Data can become stale

2. Read-Through Cache

Cache sits between app and database.

```
Read:
1. App reads from cache
2. Cache reads from DB on miss
3. Cache stores and returns data
```

Pros: - Simpler application code - Automatic cache population

Cons: - Cache must support read-through - First request always slow

3. Write-Through Cache

Data written to cache and database simultaneously.

```
Write:
1. Write to cache
2. Cache writes to DB
3. Return success
```

Pros: - Cache always consistent with DB - No stale data

Cons: - Higher write latency - Cache filled with unread data

4. Write-Behind (Write-Back) Cache

Write to cache immediately, persist to DB asynchronously.

```
Write:
1. Write to cache
2. Return success immediately
3. Async: Write to DB later
```

Pros: - Very fast writes - Reduced DB load

Cons: - Risk of data loss - Complexity in handling failures

5. Write-Around Cache

Write directly to DB, cache only on read.

```
Write:
1. Write directly to DB

Read:
1. Read from cache
2. If miss, read from DB and cache
```

Pros: - Cache not flooded with writes - Good for write-heavy workloads

Cons: - Higher read latency for new data

Cache Eviction Policies

LRU (Least Recently Used)

Evicts the least recently accessed item.

LFU (Least Frequently Used)

Evicts the least frequently accessed item.

FIFO (First In, First Out)

Evicts the oldest item.

TTL (Time To Live)

Items expire after a set time.

Random

Randomly evicts an item.

Cache Invalidation

```
1. TTL-based expiration
2. Event-based invalidation (pub/sub)
3. Write-through invalidation
4. Manual invalidation
```

Distributed Caching

Redis

- In-memory data store
- Rich data structures
- Persistence options
- Pub/Sub support
- Cluster mode

Memcached

- Simple key-value store
- Multi-threaded
- No persistence

- Simple scaling

Redis vs Memcached

Feature	Redis	Memcached
Data Types	Rich (lists, sets, hashes)	Simple (strings)
Persistence	Yes	No
Replication	Yes	No
Pub/Sub	Yes	No
Lua Scripting	Yes	No
Memory Efficiency	Lower	Higher
Multi-threaded	No (single-threaded)	Yes

---

5. Database Design

Database Types

Relational Databases (RDBMS)

- MySQL, PostgreSQL, Oracle, SQL Server
- Structured data with schemas
- ACID transactions
- SQL queries
- Joins and relationships

NoSQL Databases

Document Stores

- MongoDB, CouchDB
- JSON/BSON documents
- Flexible schema
- Nested data structures

Key-Value Stores

- Redis, DynamoDB, Riak
- Simple get/set operations
- High performance
- Limited query capabilities

Column-Family Stores

- Cassandra, HBase
- Wide-column storage
- High write throughput
- Time-series data

Graph Databases

- Neo4j, Amazon Neptune
- Nodes and relationships
- Complex relationship queries
- Social networks, recommendations

Database Indexing

B-Tree Index

- Balanced tree structure
- O(log n) search, insert, delete
- Range queries supported
- Default for most databases

```
          [50]
         /    \
     [25]       [75]
    /    \    /    \
 [10,20] [30,40] [60,70] [80,90]
```

Hash Index

- O(1) lookup
- Equality queries only
- No range queries
- Memory-based

Composite Index

Index on multiple columns.

```sql
CREATE INDEX idx_name ON users(last_name, first_name);
-- Efficient for:
--   WHERE last_name = 'Smith'
--   WHERE last_name = 'Smith' AND first_name = 'John'
-- Not efficient for:
--   WHERE first_name = 'John'
```

Full-Text Index

For text search operations.

Bitmap Index

For low-cardinality columns.

Normalization

First Normal Form (1NF)

- Atomic values (no arrays/sets)
- Each row is unique

Second Normal Form (2NF)

- 1NF + no partial dependencies
- All non-key columns depend on entire primary key

Third Normal Form (3NF)

- 2NF + no transitive dependencies
- Non-key columns depend only on primary key

Denormalization

Intentionally adding redundancy for read performance.

When to denormalize: - Read-heavy workloads - Complex joins are slow - Reporting/analytics queries

---

6. SQL vs NoSQL

When to Use SQL

- Complex queries with joins
- ACID transactions required
- Structured, stable schema
- Strong consistency needed
- Financial applications
- E-commerce orders

When to Use NoSQL

- Flexible/evolving schema
- High write throughput
- Horizontal scaling needed
- Denormalized data
- Real-time big data
- Content management
- IoT applications

Comparison Table

Aspect	SQL	NoSQL
Schema	Fixed	Dynamic
Scaling	Vertical (primarily)	Horizontal
Consistency	Strong (ACID)	Eventual (BASE)
Joins	Supported	Limited/None
Transactions	Multi-row	Limited
Query Language	SQL	Various
Best For	Complex queries	Simple queries at scale

ACID vs BASE

ACID (SQL)

- **Atomicity:** All or nothing transactions
- **Consistency:** Valid state transitions
- **Isolation:** Concurrent transactions isolated
- **Durability:** Committed data persists

BASE (NoSQL)

- **Basically Available:** System always responds
- **Soft state:** State may change over time
- **Eventual consistency:** System becomes consistent eventually

---

7. Database Scaling

Replication

Master-Slave (Primary-Replica)

```
       +---> Slave 1 (Read)
       |
Master +---> Slave 2 (Read)
(Write)|
       +---> Slave 3 (Read)
```

Synchronous Replication: - Master waits for slave acknowledgment - Strong consistency - Higher latency

Asynchronous Replication: - Master doesn't wait - Better performance - Potential data loss

Master-Master (Multi-Primary)

```
Master 1 <---> Master 2
(Read/Write)   (Read/Write)
```

Challenges: - Conflict resolution - Increased complexity - Data inconsistency risk

Sharding (Horizontal Partitioning)

Distributing data across multiple databases.

```
+-------------+     +-------------+     +-------------+
| Shard 1     |     | Shard 2     |     | Shard 3     |
| Users A-H   |     | Users I-P   |     | Users Q-Z   |
+-------------+     +-------------+     +-------------+
```

Sharding Strategies

1. Range-Based Sharding

```
Shard 1: user_id 1-1000000
Shard 2: user_id 1000001-2000000
Shard 3: user_id 2000001-3000000
```

Pros: Simple, range queries efficient **Cons:** Hotspots, uneven distribution

2. Hash-Based Sharding

```
shard_id = hash(user_id) % num_shards
```

Pros: Even distribution **Cons:** Range queries difficult, resharding complex

3. Directory-Based Sharding

Lookup table maps keys to shards.

```
+----------+----------+
| Key      | Shard    |
+----------+----------+
| user_1   | Shard 1  |
| user_2   | Shard 3  |
| user_3   | Shard 2  |
+----------+----------+
```

Pros: Flexible, easy rebalancing **Cons:** Lookup overhead, single point of failure

4. Geographic Sharding Data stored based on user location.

```
US Users -> US Shard
EU Users -> EU Shard
Asia Users -> Asia Shard
```

Sharding Challenges

1. **Joins across shards:** Complex, slow
2. **Transactions:** Distributed transactions needed
3. **Resharding:** Moving data is complex
4. **Hotspots:** Uneven data distribution
5. **Referential integrity:** Difficult to maintain

Federation (Functional Partitioning)

Split databases by function.

```
+-------------+     +-------------+     +-------------+
| Users DB    |     | Products DB |     | Orders DB   |
+-------------+     +-------------+     +-------------+
```

Pros: - Less data per database - Independent scaling - Smaller indexes

Cons: - Cross-database joins - Application complexity

---

8. Consistent Hashing

The Problem with Traditional Hashing

With traditional hashing:

```
server = hash(key) % N
```

When N changes (server added/removed), almost ALL keys are remapped.

How Consistent Hashing Works

1. Arrange servers on a hash ring (0 to 2^32-1)
2. Hash keys to points on the ring
3. Key is stored on first server clockwise from its position

```
            0
            |
     Server A(10)
           /
          /
        Key(15) -> Server B
         |
    Server B(30)
         |
         |
Server C(60)------180
         |
         |
    Server D(90)
            |
           270
```

Adding/Removing Servers

- **Adding:** Only keys between new server and previous server are remapped
- **Removing:** Only keys on removed server are remapped

Virtual Nodes

To ensure even distribution, each physical server has multiple virtual nodes.

```
Server A: Virtual nodes at positions 10, 50, 90, 130...
Server B: Virtual nodes at positions 25, 65, 105, 145...
```

Benefits: - More even distribution - Gradual rebalancing - Handles heterogeneous servers

Use Cases

- Distributed caches (Memcached, Redis Cluster)
- Load balancing
- Distributed databases (Cassandra, DynamoDB)
- CDN routing

---

9. CAP Theorem

Definition

In a distributed system, you can only guarantee 2 of 3:

- **Consistency:** All nodes see same data at same time
- **Availability:** Every request receives a response
- **Partition Tolerance:** System works despite network failures

```
      Consistency
         /\
        /  \
       /    \
      /  CP  \
     /________\
    /\        /\
   /  \  CA  /  \
  / AP \    /    \
 /______\  /______\
Availability  Partition
              Tolerance
```

Why Not All Three?

In a network partition: - To maintain **Consistency**: Must block requests (sacrifice Availability) - To maintain **Availability**: Must accept writes (sacrifice Consistency) - **Partition Tolerance**: Required in distributed systems (networks fail)

System Classifications

CP Systems (Consistency + Partition Tolerance)

- Sacrifice availability during partitions
- Examples: MongoDB, HBase, Redis

AP Systems (Availability + Partition Tolerance)

- Sacrifice consistency during partitions
- Examples: Cassandra, CouchDB, DynamoDB

CA Systems (Consistency + Availability)

- Only possible without partitions (single node)
- Examples: Traditional RDBMS

PACELC Theorem

Extension of CAP: If Partition, choose A or C; Else, choose L or C

- **PA/EL:** Availability during partition, Latency otherwise (Cassandra)
- **PA/EC:** Availability during partition, Consistency otherwise
- **PC/EL:** Consistency during partition, Latency otherwise (MongoDB)
- **PC/EC:** Consistency always (Traditional RDBMS)

---

10. Message Queues

What are Message Queues?

Asynchronous communication between services.

```
Producer --> [Message Queue] --> Consumer
```

Benefits

1. **Decoupling:** Services don't need to know about each other
2. **Scalability:** Add consumers as needed
3. **Reliability:** Messages persist if consumer fails
4. **Load leveling:** Smooth out traffic spikes

Messaging Patterns

1. Point-to-Point (Queue)

One producer, one consumer per message.

```
Producer --> [Queue] --> Consumer
```

2. Publish-Subscribe (Topic)

One producer, multiple consumers.

```
                   +--> Consumer 1
Producer --> [Topic] --> Consumer 2
                   +--> Consumer 3
```

3. Request-Reply

Synchronous-like communication.

```
Client --> [Request Queue] --> Server
Client <-- [Reply Queue] <-- Server
```

Message Queue Technologies

Apache Kafka

- Distributed streaming platform
- High throughput
- Log-based storage
- Message retention
- Partitioned topics

Use cases: Event sourcing, log aggregation, stream processing

RabbitMQ

- AMQP protocol
- Flexible routing
- Multiple protocols
- Management UI

Use cases: Task queues, RPC, pub/sub

Amazon SQS

- Fully managed
- Automatic scaling
- At-least-once delivery
- Standard and FIFO queues

Redis Pub/Sub

- In-memory
- Simple pub/sub
- No persistence

Delivery Guarantees

At-Most-Once

- Message may be lost
- No duplicates
- Fastest

At-Least-Once

- Message delivered at least once
- Possible duplicates
- Requires idempotent consumers

Exactly-Once

- Message delivered exactly once
- Most complex
- Requires idempotent operations or deduplication

Message Queue Design Considerations

```
1. Message ordering (FIFO vs unordered)
2. Message size limits
3. Retention period
4. Dead letter queue (failed messages)
5. Consumer acknowledgment
```

```
6. Idempotency
7. Message priority
8. Batching
```

---

11. Microservices Architecture

Monolithic vs Microservices

Monolithic

```
+-----------------------------------+
|          Application              |
|  +------+  +------+  +------+    |
|  | UI   |  |Logic |  | Data |    |
|  +------+  +------+  +------+    |
+-----------------------------------+
               |
        [Single Database]
```

Microservices

```
+--------+  +--------+  +--------+
| User   |  | Order  |  | Payment|
| Service|  | Service|  | Service|
+--------+  +--------+  +--------+
    |           |           |
 [User DB]  [Order DB]  [Payment DB]
```

Microservices Principles

1. **Single Responsibility:** Each service does one thing well
2. **Independently Deployable:** Deploy without affecting others
3. **Decentralized Data:** Each service owns its data
4. **Design for Failure:** Expect services to fail
5. **Automation:** CI/CD, infrastructure as code

Service Communication

Synchronous (Request/Response)

- REST APIs
- gRPC
- GraphQL

Asynchronous (Event-Driven)

- Message queues
- Event streaming
- Pub/Sub

API Gateway

Single entry point for all clients.

```
              +---> User Service
              |
Client --> API Gateway ---> Order Service
```

```
              |
              +---> Payment Service
```

Responsibilities: - Request routing - Authentication/Authorization - Rate limiting - Load balancing - Caching - Request/Response transformation - Logging and monitoring

Service Discovery

How services find each other.

Client-Side Discovery

Client queries registry and load balances.

```
Client --> [Service Registry] --> get Service B instances
Client --> Service B (instance 1)
```

Server-Side Discovery

Router/load balancer queries registry.

```
Client --> Router --> [Service Registry]
              |
              +--> Service B
```

Tools: Consul, etcd, ZooKeeper, Kubernetes DNS

Circuit Breaker Pattern

Prevent cascade failures.

```
States:
1. CLOSED: Normal operation, requests pass through
2. OPEN: Failures exceeded threshold, requests fail immediately
3. HALF-OPEN: Test if service recovered

         success
CLOSED ---------> CLOSED
   |
   | failure threshold
   v
  OPEN ---------> HALF-OPEN
         timeout      |
                      | success
                      v
                    CLOSED
```

Saga Pattern

Manage distributed transactions.

Choreography

Each service publishes events.

```
Order Service --> [Order Created Event]
                        |
                        v
Inventory Service --> [Inventory Reserved Event]
                        |
                        v
```

```
Payment Service --> [Payment Processed Event]
```

Orchestration

Central coordinator manages flow.

```
Saga Orchestrator --> Order Service
        |
        v
Saga Orchestrator --> Inventory Service
        |
        v
Saga Orchestrator --> Payment Service
```

Microservices Challenges

1. **Distributed system complexity**
2. **Data consistency**
3. **Network latency**
4. **Service discovery**
5. **Debugging and tracing**
6. **Testing**
7. **Deployment coordination**

---

12. API Design

REST (Representational State Transfer)

REST Principles

1. **Stateless:** No client context stored
2. **Client-Server:** Separation of concerns
3. **Cacheable:** Responses must define cacheability
4. **Uniform Interface:** Consistent API design
5. **Layered System:** Client doesn't know if connected directly

HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Read resource	Yes	Yes
POST	Create resource	No	No
PUT	Update/Replace resource	Yes	No
PATCH	Partial update	No	No
DELETE	Delete resource	Yes	No

REST Best Practices

```
# Use nouns, not verbs
GET /users           # Good
GET /getUsers        # Bad

# Use plural nouns
GET /users           # Good
GET /user            # Bad
```

```
# Use HTTP status codes
200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error

# Versioning
/api/v1/users
/api/v2/users

# Filtering, sorting, pagination
GET /users?status=active&sort=name&page=2&limit=20

# Nested resources
GET /users/123/orders
GET /orders/456/items
```

GraphQL

Query language for APIs.

```
# Query
query {
  user(id: "123") {
    name
    email
    orders {
      id
      total
    }
  }
}

# Response
{
  "user": {
    "name": "John",
    "email": "john@example.com",
    "orders": [
      {"id": "1", "total": 100},
      {"id": "2", "total": 250}
    ]
  }
}
```

GraphQL vs REST

Aspect	REST	GraphQL
Endpoints	Multiple	Single
Data fetching	Over/under fetching	Exact data needed
Versioning	URL versioning	Schema evolution
Caching	HTTP caching	Custom caching

Aspect	REST	GraphQL
Learning curve	Lower	Higher

gRPC

High-performance RPC framework using Protocol Buffers.

```
// user.proto
service UserService {
  rpc GetUser(GetUserRequest) returns (User);
  rpc ListUsers(ListUsersRequest) returns (stream User);
}

message User {
  string id = 1;
  string name = 2;
  string email = 3;
}
```

gRPC Features

- Binary serialization (smaller, faster)
- HTTP/2 support
- Bi-directional streaming
- Code generation
- Strong typing

When to Use gRPC

- Internal microservice communication
- Low-latency requirements
- Streaming data
- Polyglot environments

API Authentication

API Keys

Simple, passed in header or query parameter.

OAuth 2.0

Industry standard for authorization.

```
1. User authorizes app
2. App receives authorization code
3. App exchanges code for access token
4. App uses access token for API calls
```

JWT (JSON Web Tokens)

Self-contained tokens with claims.

```
Header.Payload.Signature

{
  "sub": "user123",
  "name": "John Doe",
```

```
  "exp": 1516239022
}
```

---

13. Content Delivery Networks (CDN)

What is a CDN?

Geographically distributed servers that cache content closer to users.

```
                      +---> Edge Server (US)
                      |
Origin Server --> CDN +---> Edge Server (EU)
                      |
                      +---> Edge Server (Asia)
```

How CDN Works

1. User requests content
2. Request routed to nearest edge server
3. If cached (hit): Return immediately
4. If not cached (miss): Fetch from origin, cache, return

CDN Benefits

1. **Reduced latency:** Content served from nearby server
2. **Reduced load:** Origin server handles fewer requests
3. **High availability:** Multiple edge servers
4. **DDoS protection:** Distributed infrastructure
5. **Bandwidth savings:** Cached content doesn't use origin bandwidth

CDN Content Types

Static Content

- Images, videos
- CSS, JavaScript
- HTML pages
- Downloads

Dynamic Content

- Personalized pages
- Real-time data
- Edge computing
- API responses (with short TTL)

CDN Caching Strategies

Push CDN

Origin pushes content to edge servers.

Pros: Content always available **Cons:** Storage costs, update complexity

Pull CDN

Edge servers pull content on demand.

Pros: Only requested content cached **Cons:** First request slower

CDN Providers

- Cloudflare
- AWS CloudFront
- Akamai
- Fastly
- Google Cloud CDN
- Azure CDN

CDN Configuration

```
# Cache-Control headers
Cache-Control: public, max-age=31536000
Cache-Control: private, no-cache
Cache-Control: no-store

# CDN-specific settings
- TTL (Time to Live)
- Cache key customization
- Origin failover
- Purge/invalidation
```

---

14. Proxies

Forward Proxy

Client uses proxy to access internet.

```
Client --> Forward Proxy --> Internet --> Server
```

Use cases: - Anonymous browsing - Content filtering - Bypass restrictions - Caching

Reverse Proxy

Server-side proxy that handles client requests.

```
Client --> Internet --> Reverse Proxy --> Server
```

Use cases: - Load balancing - SSL termination - Caching - Compression - Security (hide origin servers) - A/B testing

Common Reverse Proxies

Nginx

- High performance
- Load balancing
- Static content serving
- Reverse proxy

HAProxy

- TCP/HTTP load balancer
- High availability
- SSL termination

Envoy

- Modern proxy for microservices
- Service mesh
- Observability

---

15. Storage Systems

Block Storage

Raw storage volumes attached to servers.

Examples: AWS EBS, Azure Disk, Google Persistent Disk

Use cases: - Boot volumes - Database storage - High-performance apps

Object Storage

Store unstructured data as objects with metadata.

Examples: AWS S3, Google Cloud Storage, Azure Blob

Use cases: - Media files - Backups - Static websites - Data lakes

Object Storage Features

- Unlimited scalability
- HTTP access
- Metadata support
- Versioning
- Lifecycle policies

File Storage

Shared file systems accessible by multiple servers.

Examples: AWS EFS, Azure Files, Google Filestore

Use cases: - Shared application data - Content management - Home directories

Comparison

Type	Access	Scalability	Use Case
Block	Volume mount	Limited	Databases
Object	HTTP/API	Unlimited	Media, backups
File	File mount	Moderate	Shared files

Distributed File Systems

HDFS (Hadoop Distributed File System)

- Large files
- Batch processing
- Write once, read many

GlusterFS

- Scale-out file system
- Multiple protocols
- High availability

---

16. Distributed Systems Concepts

Consensus Algorithms

Paxos

- First proven consensus algorithm
- Complex to understand
- Used in Chubby, Spanner

Raft

- Understandable consensus
- Leader election
- Log replication
- Used in etcd, Consul

Raft States

```
+----------+     timeout      +----------+
| Follower | ---------------> | Candidate|
+----------+                  +----------+
     ^                             |
     |     receives majority       |
     |         votes               v
     +----------------------- +--------+
                              | Leader |
                              +--------+
```

Leader Election

Process of choosing one node to be the leader.

Methods: 1. **Bully Algorithm:** Highest ID wins 2. **Ring Algorithm:** Token passing 3. **Consensus-based:** Raft, Paxos

Heartbeat

Periodic signals to indicate node is alive.

```
Leader sends heartbeat every 150ms
If follower doesn't receive for 300ms, start election
```

Quorum

Minimum nodes needed for consensus.

```
Quorum = (N/2) + 1

For 5 nodes: Quorum = 3
For 7 nodes: Quorum = 4
```

Vector Clocks

Track causality in distributed systems.

```
Node A: [A:1, B:0, C:0]
Node B: [A:1, B:1, C:0]  (after receiving from A)
Node C: [A:1, B:1, C:1]  (after receiving from B)
```

Gossip Protocol

Information dissemination through peer-to-peer communication.

```
1. Node A has update
2. A tells random nodes B and C
3. B and C tell other random nodes
4. Eventually all nodes have update
```

Use cases: Failure detection, membership, data dissemination

Two-Phase Commit (2PC)

Distributed transaction protocol.

```
Phase 1 (Prepare):
Coordinator --> Participants: "Prepare to commit"
Participants --> Coordinator: "Ready" or "Abort"

Phase 2 (Commit):
If all ready:
  Coordinator --> Participants: "Commit"
Else:
  Coordinator --> Participants: "Abort"
```

Limitations: - Blocking protocol - Single point of failure (coordinator)

Three-Phase Commit (3PC)

Non-blocking version with pre-commit phase.

---

17. Security

Authentication vs Authorization

- **Authentication:** Who are you?
- **Authorization:** What can you do?

HTTPS/TLS

Encrypted communication.

```
1. Client hello (supported ciphers)
2. Server hello (chosen cipher, certificate)
3. Key exchange
4. Encrypted communication
```

Common Security Practices

1. Input Validation

- Sanitize all user input
- Prevent SQL injection
- Prevent XSS

2. Encryption

- Encrypt data at rest
- Encrypt data in transit
- Key management

3. Authentication

- Strong passwords
- Multi-factor authentication
- Session management

4. Authorization

- Role-based access control (RBAC)
- Principle of least privilege
- API scopes

5. Rate Limiting

- Prevent brute force
- DDoS mitigation

6. Logging and Monitoring

- Audit trails
- Anomaly detection
- Incident response

OAuth 2.0 Flows

Authorization Code Flow

For server-side apps.

Implicit Flow

For client-side apps (deprecated).

Client Credentials Flow

For machine-to-machine.

Resource Owner Password Flow

Direct username/password (legacy).

---

18. Monitoring and Logging

The Three Pillars of Observability

1. Metrics

Numerical measurements over time.

```
- Request count
- Error rate
- Response time (p50, p95, p99)
- CPU/Memory usage
- Queue depth
```

Tools: Prometheus, Datadog, CloudWatch

2. Logs

Timestamped records of events.

```
2024-01-15 10:30:45 INFO [user-service] User 123 logged in
2024-01-15 10:30:46 ERROR [order-service] Failed to process order 456
```

Tools: ELK Stack, Splunk, Loki

3. Traces

Track requests across services.

```
Request ID: abc123
  -> API Gateway (10ms)
    -> User Service (50ms)
    -> Order Service (100ms)
      -> Database (30ms)
    -> Payment Service (200ms)
Total: 390ms
```

Tools: Jaeger, Zipkin, AWS X-Ray

Key Metrics (RED Method)

- **Rate:** Requests per second
- **Errors:** Failed requests
- **Duration:** Request latency

Key Metrics (USE Method)

- **Utilization:** Resource usage percentage
- **Saturation:** Queue depth, waiting
- **Errors:** Error events

Alerting

```
Alert Rules:
- Error rate > 5% for 5 minutes
- Latency p99 > 1s for 10 minutes
- CPU > 80% for 15 minutes
- Disk > 90%

Alert Channels:
- PagerDuty
```

```
- Slack
- Email
- SMS
```

Log Aggregation Architecture

```
+--------+     +--------+     +---------+     +---------+
| Service| --> | Shipper| --> | Message | --> | Storage |
+--------+     | (Beats)|     | Queue   |     | (ES)    |
                              | (Kafka) |     +---------+
                              +---------+          |
                                                   v
                                              +----------+
                                              | Dashboard|
                                              | (Kibana) |
                                              +----------+
```

---

19. Rate Limiting

Why Rate Limiting?

1. Prevent abuse
2. Ensure fair usage
3. Protect against DDoS
4. Manage costs
5. Meet SLAs

Rate Limiting Algorithms

1. Token Bucket

```
Bucket capacity: 10 tokens
Refill rate: 1 token/second

Request arrives:
- If tokens available, consume and allow
- If no tokens, reject

Allows bursts up to bucket capacity
```

2. Leaky Bucket

```
Bucket capacity: 10 requests
Leak rate: 1 request/second

Request arrives:
- Add to bucket
- If bucket full, reject
- Process requests at constant rate

Smooths out bursts
```

3. Fixed Window Counter

```
Window: 1 minute
Limit: 100 requests
```

```
Count requests in current window
Reset count at window boundary

Problem: Burst at window boundaries
```

4. Sliding Window Log

```
Store timestamp of each request
Count requests in last window
More accurate but memory intensive
```

5. Sliding Window Counter

```
Weighted average of current and previous window
More accurate than fixed window
Less memory than sliding log
```

Rate Limiting Strategies

User-based

Limit per user/API key.

IP-based

Limit per IP address.

Endpoint-based

Different limits per endpoint.

Tiered

Different limits per plan/tier.

Implementation

```javascript
// Redis rate limiter example
async function isRateLimited(userId, limit, windowSecs) {
    const key = `rate_limit:${userId}`;
    const current = await redis.incr(key);

    if (current === 1) {
        await redis.expire(key, windowSecs);
    }

    return current > limit;
}
```

HTTP Headers

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1640000000
Retry-After: 60
```

---

20. Back-of-the-Envelope Calculations

Important Numbers to Know

Storage

```
1 Byte = 8 bits
1 KB = 1,000 Bytes
1 MB = 1,000 KB
1 GB = 1,000 MB
1 TB = 1,000 GB
1 PB = 1,000 TB
```

Time

```
1 second = 1,000 milliseconds
1 millisecond = 1,000 microseconds
1 microsecond = 1,000 nanoseconds
```

Latency Numbers

```
L1 cache reference:                 0.5 ns
L2 cache reference:                   7 ns
Main memory reference:              100 ns
SSD random read:                 16,000 ns  (16 us)
HDD seek:                     2,000,000 ns  (2 ms)
Send packet CA->NL->CA:     150,000,000 ns  (150 ms)
```

Traffic Estimates

```
1 million requests/day = ~12 requests/second
10 million requests/day = ~120 requests/second
100 million requests/day = ~1,200 requests/second
1 billion requests/day = ~12,000 requests/second
```

Quick Conversions

```
Seconds in a day: 86,400 (~100,000)
Seconds in a year: 31,536,000 (~30 million)
2^10 = 1,024 (~1,000)
2^20 = 1,048,576 (~1 million)
2^30 = ~1 billion
2^40 = ~1 trillion
```

Example Calculation: Twitter-like System

Requirements: - 300 million monthly active users - 50% daily active - Each user makes 2 tweets/day - Each user reads 100 tweets/day - Tweet size: 140 chars + metadata = 500 bytes

Calculations:

```
Daily active users: 150 million
Tweets per day: 150M * 2 = 300M tweets
Write QPS: 300M / 86,400 = ~3,500/sec
Peak QPS: 3,500 * 3 = ~10,500/sec

Reads per day: 150M * 100 = 15 billion
Read QPS: 15B / 86,400 = ~175,000/sec

Daily storage: 300M * 500 bytes = 150 GB
```

```
Yearly storage: 150 GB * 365 = ~55 TB
```

---

21. System Design Interview Framework

Step 1: Requirements Clarification (5 minutes)

Ask questions to understand scope:

Functional Requirements

- What features are needed?
- Who are the users?
- What are the use cases?

Non-Functional Requirements

- Scale: How many users? Requests per second?
- Performance: Latency requirements?
- Availability: What uptime is needed?
- Consistency: Strong or eventual?

Constraints

- Budget
- Timeline
- Technology constraints

Step 2: Back-of-Envelope Estimation (5 minutes)

Calculate: - Traffic estimates (QPS, bandwidth) - Storage requirements - Memory requirements (cache size)

Step 3: High-Level Design (10 minutes)

Draw main components: - Clients - Load balancers - Application servers - Databases - Caches - Message queues

Step 4: Deep Dive (15 minutes)

Detail critical components: - Data model - API design - Database schema - Algorithm choices

Step 5: Identify Bottlenecks (5 minutes)

Discuss: - Single points of failure - Scaling strategies - Performance optimizations - Monitoring and alerts

Common Mistakes

1. **Jumping into solution:** Not gathering requirements
2. **Ignoring scale:** Not doing calculations
3. **Over-engineering:** Adding unnecessary complexity
4. **Under-engineering:** Missing critical components
5. **Not discussing trade-offs:** Every decision has trade-offs
6. **Monologue:** Not engaging with interviewer

Interview Tips

1. **Think out loud:** Share your thought process
2. **Start simple:** Begin with MVP, then iterate
3. **Ask clarifying questions:** Show analytical thinking
4. **Discuss trade-offs:** Show depth of knowledge
5. **Use real examples:** Reference known systems
6. **Be open to hints:** Interviewer is trying to help
7. **Practice:** Do mock interviews

Quick Reference Tables

Choosing the Right Database

Need	Choice
Complex queries, transactions	PostgreSQL, MySQL
Document storage	MongoDB, CouchDB
High write throughput	Cassandra, ScyllaDB
Caching	Redis, Memcached
Search	Elasticsearch
Time series	InfluxDB, TimescaleDB
Graph relationships	Neo4j

Choosing the Right Cache

Need	Choice
Simple key-value	Memcached
Rich data types	Redis
Persistent cache	Redis
Session storage	Redis
Rate limiting	Redis

Choosing Communication Pattern

Need	Pattern
Synchronous, simple	REST
Flexible queries	GraphQL
High performance	gRPC
Asynchronous	Message Queue
Real-time	WebSocket

End of System Design Concepts Guide